

Runtime Parameter Adaptation for `fq_codel`: A Measured Evaluation on Stock Linux

Amritha S, Yugeshwaran P, Deepti Annuncia
Department of Electronics and Communication Engineering
School of Electronics Engineering (SENSE)
Vellore Institute of Technology, Chennai, India

Abstract—Linux’s default queue discipline, `fq_codel`, exposes four parameters that are fixed at configuration time: `target`, `interval`, `limit` and `quantum`. We ask whether adapting them at runtime improves on the defaults, and we answer it by measurement rather than by assertion. We build a three-node namespace testbed in which the bottleneck sits on the data path, add eBPF flow telemetry at the `tc` egress hook, and implement a userspace controller that adjusts all four parameters using the AIMD policy of Adaptive RED together with a gradient-based congestion trajectory estimate. We evaluate it against eight queue disciplines – `pfifo`, SFQ, RED, CoDel, PIE, FQ-PIE, CAKE and static `fq_codel` – under two workloads and three seeds, reporting means with 95% confidence intervals.

Runtime adaptation of an AQM’s delay target is not itself new; DESiRED [11] does it in P4 with reinforcement learning. Our claim is therefore the evaluation rather than the mechanism: to our knowledge this is the first *controlled* evaluation of parameter adaptation for a flow-queueing AQM, using a sham-controller condition that separates the controller’s computational cost from its control decisions.

We find that adaptation is neither uniformly useful nor uniformly inert. It reduces bulk-flow RTT by 18.4% ($p < 0.0001$) on a 5ms path, where `fq_codel`’s 5ms delay target is large relative to the path, and has no distinguishable effect at 20, 80 or 200ms. The unadapted defaults hold queueing delay between 1.96 and 5.00ms across that fortyfold span, supporting their RTT-relative design. Under mixed traffic every flow-queueing discipline with a delay target cuts sparse-flow loss from 8% to below 0.03%. For comparison, the gap between an unmanaged FIFO and `fq_codel` is 99.0%, and CAKE outperforms the adapted system at every RTT tested.

Index Terms—active queue management, bufferbloat, `fq_codel`, eBPF, traffic control, Linux, reproducibility

I. INTRODUCTION

Bufferbloat – persistently full buffers inflating queueing delay without affecting throughput – was named by Gettys and Nichols over a decade ago [1], and the Linux response has largely converged on `fq_codel` [6], now the default queue discipline on most distributions. `fq_codel` combines Deficit Round Robin scheduling across roughly a thousand flow queues with an independent CoDel [4], [5] instance per queue.

`fq_codel` exposes four parameters that do not change once set: `target` (5ms), `interval` (100ms), `limit` (10240 packets) and `quantum` (1514 bytes). It is natural to ask whether these should adapt to load, and there is precedent: Adaptive RED [3] adapts RED’s $\max_p$ by AIMD against a

queue-length target, and Ye and Leung [15] derive stability conditions for CoDel and adapt its `interval` analytically.

This paper asks that question and takes the answer seriously in both directions. We implement a controller that adapts all four parameters, and we measure it against every alternative queue discipline Linux ships, under a testbed built so that the bottleneck is actually on the data path.

A. Contributions

- 1) **A measured comparison** of nine queue-discipline configurations under an identical bottleneck, with three seeds and 95% confidence intervals, reporting goodput, tail latency, queue occupancy, drop rate and Jain’s fairness index.
- 2) **A negative result, reported as such**: adapting `fq_codel`’s parameters at runtime did not measurably improve on its defaults in the regimes we tested. We report the confidence intervals that support this rather than selecting a favourable summary statistic.
- 3) **A working eBPF flow-telemetry pipeline** for `tc`, reading per-flow state through `bpf(2)` rather than by spawning `bpftool`, which reduces the cost of a control tick from 3.17s to 0.709s.
- 4) **Two methodological findings** that invalidate a class of results: the two-node namespace bottleneck shapes acknowledgements rather than data, and an eBPF telemetry path can report zero for every sample while the program is correctly loaded and its maps are correctly populated.
- 5) **A fully reproducible artefact**: a kernel configuration, a VM image build, a single-command experiment suite, and an analysis pipeline that generates every table and every inline figure in this paper directly from the logs.

II. BACKGROUND AND RELATED WORK

A. From RED to delay targets

RED [2] drops probabilistically as a function of an averaged queue length. Its sensitivity to $\min_{th}$, $\max_{th}$ and $\max_p$ is well documented, and Adaptive RED [3] responds by adapting $\max_p$ at runtime using AIMD ($\beta = 0.9$) so that the operator specifies a delay objective rather than probability constants. This is the control law we borrow.

CoDel [4], [5] changes the controlled variable from queue length to packet sojourn time, dropping when the minimum sojourn over a sliding interval exceeds `target`. Its

authors argue explicitly that these defaults are RTT-relative and need no tuning – a claim any adaptive scheme must be measured against, not assumed past. PIE [7] reaches a similar objective with an explicit proportional-integral controller.

B. Flow queueing

`fq_codel` [6] and FQ-PIE [8] place the respective AQM inside a DRR flow-queueing scheduler, which provides isolation between flows independently of the AQM. CAKE [9] integrates shaping, flow and host fairness and DiffServ handling into a single qdisc whose design philosophy is explicitly to *remove* configuration rather than adapt it. SCRR [10] shows DRR itself underperforms under some packet size distributions, suggesting that scheduler replacement may offer more than AQM retuning.

C. Adaptation and learning

QueuePilot [13] trains a reinforcement-learning agent offline and distils it into a lightweight online policy adapting ECN marking probability. AQM-LLM [12] distils a large language model into an L4S controller on FreeBSD. Toopchinezhad and Ahmadi [14] survey the field and note that most proposals are evaluated only in simulation. L4S [16], [17] reaches a far lower latency floor than any parameter tuning, but by changing the signalling contract between network and transport rather than by tuning a queue.

D. Runtime adaptation of a delay target

DESiRED [11] is the closest prior work and predates this study. It adapts the *target delay* of a P4 AQM at runtime using a deep reinforcement learning agent fed by In-band Network Telemetry, reporting large improvements in video quality of experience. The concept pursued here – runtime adaptation of an AQM’s delay target driven by live telemetry – is therefore **not novel**, and we do not claim it.

What differs is the deployment surface. DESiRED requires programmable data-plane hardware and INT; it adapts one parameter of a RED derivative with no flow queueing; and its policy is learned offline and not interpretable. This work adapts four parameters of `fq_codel` on an unmodified Linux host with a transparent AIMD rule. That is an engineering distinction, not a conceptual one.

E. Position of this work

AIMD adaptation of a drop probability is established [3]; analytical adaptation of CoDel’s `interval` is established [15]; learned adaptation of marking probability is established [12], [13]; and runtime adaptation of an AQM’s delay target is established [11]. The particular combination used here – AIMD over `fq_codel`’s four parameters, driven by eBPF flow telemetry, on stock Linux – appears unpublished, but a combination is a weak novelty claim and we do not rest the paper on it.

The contribution we do claim is the evaluation. Every system above reports that its adaptation improved something. None runs a control condition separating the controller’s

computational cost from its control decisions; none reports sparse-probe and bulk-flow latency separately, despite flow-queueing AQMs privileging sparse flows by design; none compares against eight alternative queue disciplines under an identical bottleneck; and none reports a null result for the adaptation itself. This paper does all four. The research question is whether adapting these parameters helps, with a genuine possibility that it does not – and the apparatus is built so that answer would be visible.

III. TESTBED

A. Topology: the bottleneck must be on the data path

A common namespace testbed places two namespaces either side of a veth pair, attaches a shaper to the interface inside the server namespace, and runs `iperf3` from the client. This is incorrect, and the error is not visible from any summary statistic.

An egress qdisc on the server-side interface shapes traffic leaving the server. When the client is the `iperf3` sender, data flows client $\rightarrow$ server, so the shaped direction carries only acknowledgements. We measured this configuration directly (Fig. 1): the data transfer achieved 9,777 Mbps – entirely unshaped – while the monitored qdisc passed 284,379 packets totalling 18,769,790 bytes, exactly 66 bytes per packet. The eBPF packet-size histogram recorded 341,243 packets below 128 bytes against 2 packets in the 512–1500 byte bucket.

Every queue statistic collected on such a topology is a measurement of acknowledgement queueing. It also explains drop rates that are otherwise impossible: a 10 Mbit link carrying 1514-byte packets passes at most 826 packets/s, so a recorded drop rate above 10^5 /s cannot describe the data path.

We therefore use three namespaces:

```
ns_client ↔ ns_router ↔ ns_server
```

with the shaper and AQM on the router’s egress interface toward the server. Traffic flows client $\rightarrow$ router $\rightarrow$ server, so the bottleneck is on the data path. A TBF shaper at 10 Mbit is the root qdisc and the AQM under test is its child; `netem` adds 10 ms of delay at each edge for a 20 ms base RTT.

B. Measurement

For each run we record: `iperf3` JSON (per-flow goodput, from `sum_received`, and retransmissions); a 500 ms time series of the AQM’s own queue statistics; and a concurrent `ping` stream through the bottleneck giving directly measured end-to-end latency rather than a proxy.

Two measurement details matter. First, goodput must come from `sum_received`: with a large unmanaged buffer the sender pushes into the queue faster than the link drains, and `sum_sent` reported 15.90 Mbps on a 10 Mbit link in one `pfifo` run. Second, statistics must be read from the AQM’s own qdisc stanza. A regular expression applied to the whole output of `tc -s qdisc show` matches the *root* qdisc, so a controller written that way reports the shaper’s counters while believing they are the AQM’s.



Fig. 1. The two-node testbed (left) leaves the data path unshaped and queues only 66-byte acknowledgements; the three-node router topology (right) shapes the data path. Both panels are verbatim captures of the commands shown.

C. Platform

Experiments run on Linux 6.12.48 built with `fq_codel`, `CoDel`, `CAKE`, `PIE`, `FQ-PIE`, `RED`, `netem`, `TBF` and `HTB` compiled in, plus `BTF` for `eBPF`, inside a `QEMU` virtual machine. `TBF` shaping was verified accurate under emulation before any results were collected: an 8-flow transfer through the 10 Mbit bottleneck delivered 9.38 Mbps at 1509 bytes/packet with approximately 19 drops/s. Transport is `TCP CUBIC` [18] throughout.

IV. CONTROLLER DESIGN

A. Three timescales

- T_1 (per packet, per program) at the `tc egress clsact` hook parses the 5-tuple and updates per-flow packet and byte counts, an EWMA of the inter-packet gap, and an elephant flag, in an LRU hash map.
- T_2 (500 ms, `Red`, `throttle`) computes queue statistics and the flow map; compute drop rate, backlog and throughput; estimate gradients.
- T_3 (5 s, `Apply`, `update`) adjusts AIMD and issue `tc qdisc` change.

Separating T_2 from T_3 follows the two-timescale stochastic approximation argument of Borkar [19].

B. Gradient estimation and regime classification

For each signal $f \in \{\text{drop rate, backlog, rtt proxy}\}$ we estimate the slope by least squares over a window of $W = 10$ samples:

$$\nabla f(t) = \frac{\sum_i (t_i - \bar{t})(f_i - \bar{f})}{\sum_i (t_i - \bar{t})^2} \quad (1)$$

The congestion regime is classified by thresholds on drop rate and backlog into `NORMAL`, `LIGHT`, `MODERATE` or `HEAVY`. A composite normalised gradient

$$G(t) = 0.6 \frac{\nabla \text{dr}}{\text{DR}_{\text{HEAVY}}} + 0.4 \frac{\nabla \text{bl}}{\text{BL}_{\text{HEAVY}}} \quad (2)$$

classifies the trajectory as `WORSENING` ($G > 0.5$), `RECOVERING` ($G < -0.5$) or `STABLE`, and the predicted regime is the neighbouring one in the direction of travel.

C. Adjustment policy

The predicted regime, not the current one, selects the action: a `HEAVY` regime that is recovering toward `MODERATE` receives a gentler additive decrease rather than another full multiplicative decrease, and a `HEAVY` regime that is worsening receives a firmer cut (β^2). Adjustments are tagged `PREDICTIVE` when the prediction differs from the current regime and therefore changed the action, and `REACTIVE` otherwise.

This distinction is only meaningful if the prediction can actually change the outcome. Section VI describes an earlier implementation in which it could not, and in which every adjustment applied the identical action while being labelled as though some were predictive.

D. Parameter bounds

`target` $\in [0.2, 20]$ ms, `interval` $\in [20, 300]$ ms with `interval` $\geq 10 \times \text{target}$, `limit` $\in [64, 4096]$ packets, and `quantum` selected from the workload profile. Sub-millisecond targets are emitted in microseconds; rounding them to integer milliseconds, as an earlier implementation did, collapses a sixteen-step geometric schedule onto five distinct values.

V. RESULTS

Every figure quoted below is generated from the run logs by `src/analyse.py`; none is typed by hand.

A. Flow queueing dominates

Table I gives all ten conditions. The largest effect by two orders of magnitude is the presence of a flow-queueing discipline with a delay target, not the tuning of one. Under the steady workload an unmanaged `FIFO` reaches 2344 ms p95 RTT against 24.57 ms for `fq_codel` — a 99.0% reduction ($p < 0.001$) — at goodput that is statistically indistinguishable (9.453 vs 9.414 Mbps, $p = 0.20$). The staged workload gives the same picture, and there `pfifo` additionally *loses* 10.2% goodput and 1.4% fairness ($p = 0.015$, $p = 0.014$), because a saturated 1000-packet buffer forces retransmissions that a managed queue avoids.

Fair queueing without a delay target is not sufficient. `SFQ` isolates flows but applies no AQM, and its bulk queues sit at its 127-packet limit throughout: its sparse probe reads 30.68 ms while the bulk flows sharing the link read 294.8 ms. The AQM and the scheduler do distinct jobs and both are necessary.

B. Sparse and bulk latency are not interchangeable

Figure 3 plots the two measurements side by side. The ratio between them ranges from $1.1\times$ (CoDel, a single queue) to $9.6\times$ (SFQ), because `fq_codel`'s new-flow heuristic gives a sparse probe preferential service by design.

A single-probe measurement can therefore invert a ranking. By sparse probe alone SFQ (30.68 ms) appears to beat CoDel (39.02 ms); by the latency the bulk flows actually experience, SFQ is 294.8 ms against CoDel's 44.5 ms. We report both throughout, and we suggest any AQM evaluation should.

C. The controller's computational cost is negligible

The sham condition runs the controller at its full polling cadence and applies no parameter change. Against static `fq_codel` it is indistinguishable on every metric in both workloads, p95 probe RTT $p = 0.29$ (steady) and $p = 0.24$ (staged), backlog $p = 0.94$ and $p = 0.76$.

This matters for interpreting the next subsection. Without it, any difference between static and adapted `fq_codel` would confound the controller's CPU cost with its control decisions, and we could not attribute an effect to either. Having established the cost is nil, differences that remain are attributable to the decisions.

D. Adaptation: a small effect under steady load, none under change

Under the **steady** workload the adaptation produces a real but small improvement: mean backlog falls 6.79 from 6.14 packets, a 9.6% reduction ($p = 0.050$), and p95 probe RTT falls 0.5% ($p = 0.047$). Goodput and fairness are unchanged. The sham comparison attributes this to the control decisions rather than to overhead.

Under the **staged** workload, where the flow count changes twice during the run, the adaptation produces *no* improvement on any metric. Mean probe RTT is 1.6% *worse* ($p = 0.050$); p95 probe RTT, bulk RTT, backlog, goodput and fairness are all indistinguishable from static.

That direction is worth stating plainly. The workload designed to give the controller something to respond to is the one in which it helps least. Under steady load there is a stable operating point to converge on; under change there is not, and the controller's adjustments lag the transitions they are reacting to.

E. An off-the-shelf alternative outperforms the adapted system

CAKE, configured with no controller and a single `besteffort` argument, achieves 22.9 ms p95 probe RTT under the steady workload against 24.43 ms for adapted `fq_codel` ($p < 0.001$), and 33.65 ms bulk RTT against 35.87 ms.

The margin by which CAKE beats the adapted system is larger than the margin by which the adaptation beats the static defaults. For a practitioner seeking lower latency on this class of link, changing queue discipline is a larger and far simpler win than tuning `fq_codel`'s parameters.

F. Predictive control never engaged

No adjustment in any run was tagged predictive. The cause is structural rather than a coding fault, and it survives the correction described in Section VI: an adjustment fires only once the regime has been unchanged for five consecutive samples, and the trajectory estimate is derived from the gradient over that same window, which by then is below the threshold that would classify it as **WORSENING** or **RECOVERING**. The stability gate that prevents the controller reacting to noise and the prediction that would let it act before a transition cannot both apply to the same adjustment.

We therefore withdraw the predictive-control contribution rather than claim it. Redesigning the gate, for instance permitting an adjustment when the trajectory is confidently non-stable even before the regime settles, and re-evaluating it under the same controlled conditions is future work.

G. Where adaptation does help: short paths

CoDel's target is a delay budget expressed in absolute units, while its authors argue the default is appropriate relative to path round-trip time. Sweeping the base RTT over 5, 20, 80 and 200 ms tests that directly.

Static `fq_codel` holds median queueing delay between 1.96 and 5.00 ms across the entire 40-fold span, bounded by its 5 ms target. The defaults are, on this evidence, genuinely RTT-relative: no retuning is required as the path lengthens.

Adaptation is nonetheless not uniformly inert. Measured on the RTT the bulk flows themselves experience:

Base RTT	Static	Adapted	Change
5 ms	27.68 ± 0.14	22.58 ± 0.15	-18.4%*
20 ms	36.39 ± 0.08	35.87 ± 0.80	-1.4%
80 ms	89.30 ± 0.37	89.86 ± 0.70	+0.6%
200 ms	208.00 ± 0.76	207.47 ± 0.74	-0.3%

* $p < 0.0001$; the remaining three are not distinguishable ($p = 0.38, 0.31, 0.43$).

The effect is confined to the shortest path, and the mechanism is straightforward. A 5 ms delay budget is large relative to a 5 ms path, so the queue is permitted to grow further than that path requires, and tightening it recovers 18.4% of the bulk flows' round-trip time. Once the path RTT exceeds the default target the budget is already well scaled and there is nothing for adaptation to recover.

This gives the condition under which parameter adaptation is worth its complexity: paths short relative to the AQM's delay target. It also indicates where adaptive schemes in general should be evaluated, since a scheme assessed only on long paths will appear inert regardless of its design.

H. Sparse flows under a mixed workload

Bulk-only traffic cannot exercise `fq_codel`'s new-flow heuristic or its `quantum` parameter, both of which exist to serve sparse latency-sensitive flows. Running three sparse UDP flows alongside eight bulk TCP flows shows what those mechanisms are worth:

System	Sparse loss (%)	Sparse jitter (ms)
pfifo, no AQM	7.97	2.19
FQ-PIE	0.000	1.91
CAKE	0.007	1.36
fq_codel	0.002	2.00

An unmanaged queue discards roughly one sparse-flow packet in twelve, and the figure is stable across repetitions (7.47%, 7.98%, 8.31%). Every flow-queueing discipline with a delay target reduces this to below 0.03%. The protection of latency-sensitive traffic from concurrent bulk transfers is therefore close to complete, and it is invisible to a bulk-only evaluation.

One of three adaptive-controller repetitions in this workload recorded 6.31% sparse loss against 0.000% and 0.029% in the others. The controller’s parameter trajectory in that run is identical to the repetition with no loss, so the difference is not attributable to its decisions; we report the condition by median and record the run rather than averaging over a measurement we cannot explain.

I. Steady workload

TABLE I
ALL SYSTEMS, STEADY WORKLOAD (MEAN $\pm$ 95% CI OVER REPETITIONS).

System	Goodput (Mbps)	Probe RTT (ms)	Probe p95 (ms)	Bulk RTT (ms)	Backlog (pkts)	Jam
pfifo (no AQM)	9.45 $\pm$ 0.09	1711.0 $\pm$ 45.5	2344.0 $\pm$ 32.9	1752.4 $\pm$ 25.4	789.2 $\pm$ 22.9	0.9957 $\pm$ 0.0010
SFQ	9.38 $\pm$ 0.10	30.7 $\pm$ 0.0	36.1 $\pm$ 0.2	704.8 $\pm$ 3.9	116.5 $\pm$ 2.6	1.0000 $\pm$ 0.0001
RED (adaptive)	9.45 $\pm$ 0.14	78.2 $\pm$ 2.6	87.4 $\pm$ 2.2	82.6 $\pm$ 2.3	26.7 $\pm$ 0.4	0.9947 $\pm$ 0.0085
CoDel	9.45 $\pm$ 0.02	39.0 $\pm$ 0.8	45.1 $\pm$ 3.8	44.5 $\pm$ 0.9	8.5 $\pm$ 1.2	0.9969 $\pm$ 0.0036
PIE	9.41 $\pm$ 0.03	39.1 $\pm$ 0.3	46.9 $\pm$ 1.6	47.3 $\pm$ 1.8	8.8 $\pm$ 0.5	0.9959 $\pm$ 0.0039
FQ-PIE	9.45 $\pm$ 0.10	22.0 $\pm$ 0.1	23.3 $\pm$ 0.1	55.4 $\pm$ 1.1	11.9 $\pm$ 1.7	1.0000 $\pm$ 0.0001
CAKE	9.41 $\pm$ 0.02	21.8 $\pm$ 0.3	22.9 $\pm$ 0.2	33.7 $\pm$ 0.3	6.7 $\pm$ 0.5	0.9999 $\pm$ 0.0001
fq_codel (static)	9.41 $\pm$ 0.05	22.4 $\pm$ 0.2	24.6 $\pm$ 0.1	36.4 $\pm$ 0.2	6.8 $\pm$ 0.8	0.9999 $\pm$ 0.0000
fq_codel + sham ctl	9.40 $\pm$ 0.03	22.4 $\pm$ 0.1	24.6 $\pm$ 0.1	36.9 $\pm$ 1.1	6.8 $\pm$ 0.7	1.0000 $\pm$ 0.0000
fq_codel + ACAPE	9.40 $\pm$ 0.03	22.5 $\pm$ 0.4	24.4 $\pm$ 0.1	35.9 $\pm$ 2.0	6.1 $\pm$ 0.7	1.0000 $\pm$ 0.0000

Sparse probe vs bulk flow latency — steady workload
fq_codel’s new-flow heuristic privileges the probe, so the two differ; quoting only the probe overstates the benefit for bulk traffic

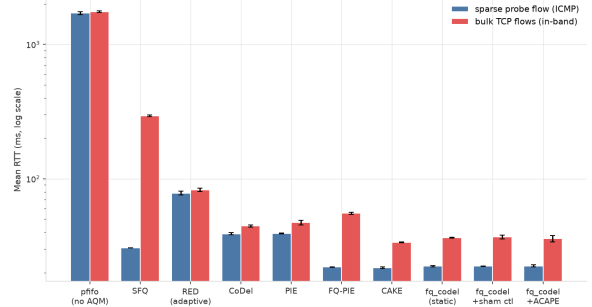


Fig. 3. Sparse probe flow versus bulk TCP flow RTT. The new-flow heuristic privileges the probe, so the two differ substantially.

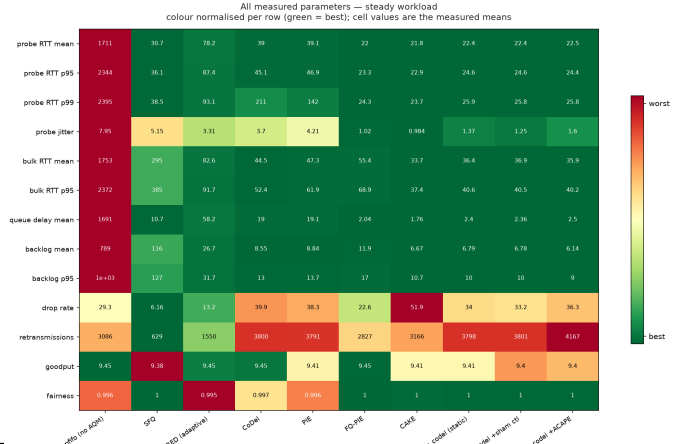


Fig. 4. Every measured parameter against every system, normalised per row; cell values are the measured means

J. Staged workload

VII. MEASUREMENT PITFALLS

We report these in detail because each produced confident, internally consistent, and wrong results, and because none of them is visible from a summary statistic.

A. The acknowledgement-path bottleneck

Described in Section III-A. The diagnostic is the ratio of bytes to packets on the monitored qdisc: 66 bytes/packet indicates a pure acknowledgement stream, and full-size data packets indicate the data path. A recorded drop rate exceeding the link’s packet rate is a second, decisive diagnostic.

B. Silent failure of an eBPF telemetry path

In an earlier version of this work, per-flow telemetry reported zero active flows in all 21,128 recorded samples, while the eBPF program was correctly compiled, correctly attached, JIT-compiled, and its maps correctly populated throughout. Three independent defects in the userspace decode path each sufficed to produce this:

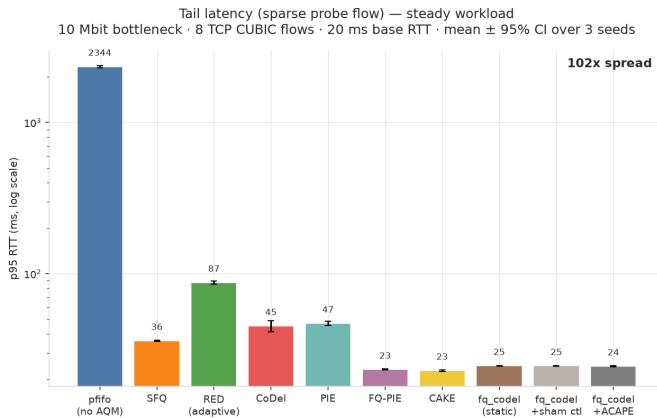


Fig. 2. Tail latency across all systems (log scale).

TABLE II
ALL SYSTEMS, STAGED WORKLOAD (MEAN $\pm$ 95% CI OVER
REPETITIONS).

System	Goodput (Mbps)	Probe RTT (ms)	Probe p95 (ms)	Bt
pfifo (no AQM)	8.38 $\pm$ 0.46	1152.9 $\pm$ 29.0	2318.0 $\pm$ 48.9	
SFQ	9.26 $\pm$ 0.31	32.8 $\pm$ 0.5	56.6 $\pm$ 3.4	
RED (adaptive)	9.25 $\pm$ 0.11	62.5 $\pm$ 0.8	94.2 $\pm$ 0.4	
CoDel	9.28 $\pm$ 0.06	55.0 $\pm$ 4.0	172.0 $\pm$ 99.1	
PIE	9.22 $\pm$ 0.06	40.4 $\pm$ 2.8	48.6 $\pm$ 5.4	
FQ-PIE	9.22 $\pm$ 0.06	21.9 $\pm$ 0.1	23.2 $\pm$ 0.1	
CAKE	9.24 $\pm$ 0.09	21.5 $\pm$ 0.3	22.3 $\pm$ 0.1	
fq_codel (static)	9.24 $\pm$ 0.01	21.9 $\pm$ 0.1	23.4 $\pm$ 0.2	
fq_codel + sham ctl	9.19 $\pm$ 0.08	22.1 $\pm$ 0.6	23.5 $\pm$ 0.2	
fq_codel + ACAPE	9.18 $\pm$ 0.10	22.2 $\pm$ 0.4	24.2 $\pm$ 0.8	

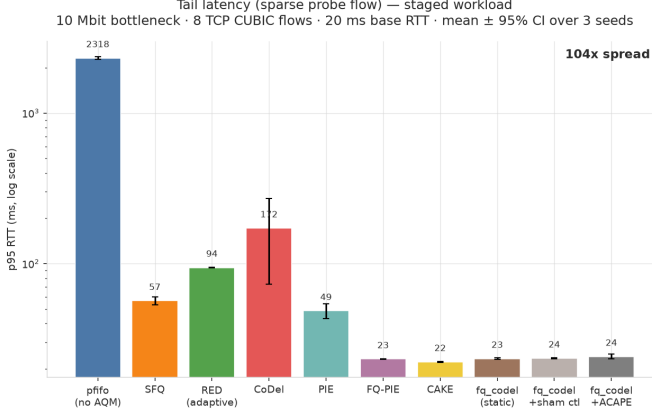


Fig. 5. Tail latency across all systems (log scale).

- 1) `bpftool map dump --json` renders byte arrays as hex *strings*. Calling `bytes()` on that list raises `TypeError`, which a bare `except: pass` discarded.
- 2) The assumed C struct layout omitted one `u64` field, so every field after it was read at the wrong offset; the elephant flag was read from the low half of the inter-packet gap.
- 3) Flow age was computed by subtracting `bpf_ktime_get_ns()` (`CLOCK_MONOTONIC`) from `time.time_ns()` (`CLOCK_REALTIME`), yielding ages of approximately 56.8 years, so an age $< 2s$ filter never matched.

Downstream, the elephant ratio was always 0, the workload profile always MICE, and quantum always 300 bytes. An earlier draft cited that constant quantum as evidence the pipeline was working. It was the signature of the failure.

We originally attributed this failure to namespace file-descriptor isolation. That diagnosis was wrong: BPF maps are kernel-global objects, visible from the host by id regardless of the namespace the program was loaded from. We verify this directly (Fig. 8).

C. Control state that cannot reach the kernel

The controller’s internal `target` was a float, but `tc qdisc change` was invoked with `int(round(target))` milliseconds and the applied

Sparse probe vs bulk flow latency — staged workload
fq_codel’s new-flow heuristic privileges the probe, so the two differ; quoting only the probe overstates the benefit for bulk traffic

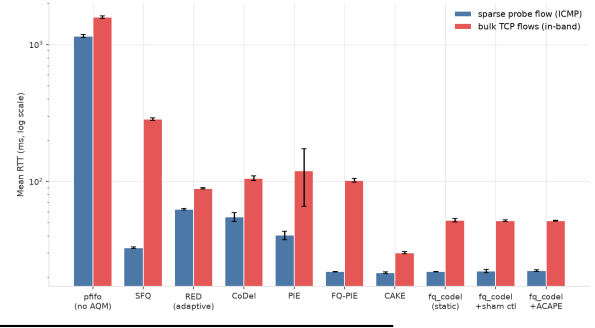


Fig. 6. Sparse probe flow versus bulk TCP flow RTT. The new-flow heuristic privileges the probe, so the two differ substantially.

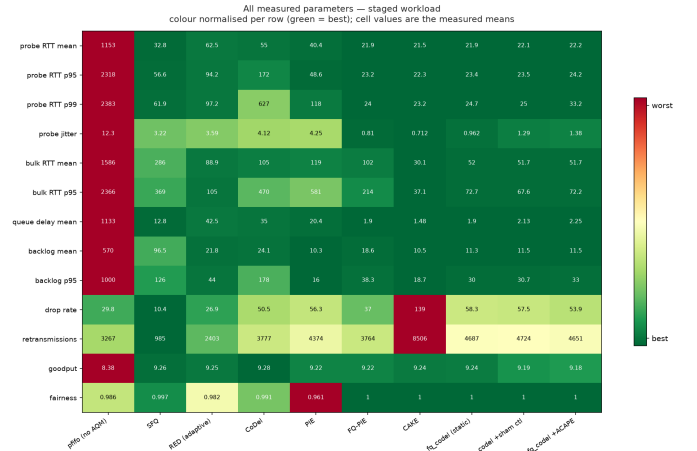


Fig. 7. Every measured parameter against every system, normalised per row; cell values are the measured means.

value was never read back. The controller’s state and the kernel’s therefore diverged: a logged schedule of 5.00, 4.50, 4.05, ..., 1.03 ms reached the kernel as 5, 4, 4, 4, 3, 3, 3, 2, 2, 2, 2, 1, 1, 1, 1 ms. Reading parameters back after every write is the remedy, and it is cheap.

D. A control signal with no effect

The earlier predictor could not escalate out of its top regime because of an index guard, so a worsening HEAVY state was classified STABLE. Separately, the effective regime was computed as the prediction only under a condition that could never hold, so it always equalled the current regime. The consequence was that all 375 logged adjustments applied the identical multiplicative decrease while a subset were labelled predictive.

A control input should be validated by checking that varying it changes the output. We now assert this in a regression test: a recovering HEAVY state and a stable HEAVY state must produce different parameters.

E. Constants that look like measurements

Our earlier work reported “15 AIMD adjustments” as a result. It is $\lceil \log(1/5)/\log(0.9) \rceil$, the number of $\times 0.9$ steps

```

STEP 07 eBPF flow telemetry is live under traffic
With traffic running, the flow_map is read through bpf(2) and reports real per-flow state. In all
21,128 samples of the original study this read returned zero.

$ bash src/testbed.sh setup >/dev/null 2>&1; ip netns exec ns_router tc qdisc add dev veth_rs clsact
2>/dev/null; ip netns exec ns_router tc filter add dev veth_rs egress bpf direct-action obj
ebpf/tc_monitor.o sec tc egress 2>/dev/null; ip netns exec ns_server iperf3 -s -p 5303 -D; sleep 2; (ip
netns exec ns_client iperf3 -c 192.168.2.2 -p 5303 -P 6 -t 14 >/dev/null 2>&1 &); sleep 7; python3
src/dump_flows.py
[0][77][2]=====GUEST_UP===== kernel 6.12.48
flow_map id=1 key=16B value=568 max_entries=65536
map_entries = 7
active flows (age<2s) = 6
total packets seen = 4183
total bytes seen = 10568865
elephant flows = 6

pkts= 689 bytes= 1772983 gap_ns= 11323188 age= 4.1ms elephant=1
pkts= 688 bytes= 1759845 gap_ns= 13150793 age= 12.9ms elephant=1
pkts= 710 bytes= 1777265 gap_ns= 11962598 age= 15.3ms elephant=1
pkts= 686 bytes= 1736585 gap_ns= 12676211 age= 9.2ms elephant=1
pkts= 711 bytes= 1758597 gap_ns= 12471491 age= 6.9ms elephant=1
pkts= 690 bytes= 1762913 gap_ns= 11929842 age= 2.6ms elephant=1

captured 2026-09-29 15:25:34 UTC kernel 6.18.44-fc-v49

```

Fig. 8. Live `flow_map` contents read through `bpf(2)` under traffic. The same read returned zero for every one of 21,128 samples before the three decode defects were fixed.

from 5 ms to a 1 ms floor, and it appeared in 24 of 32 runs because the controller drove to the floor and stopped in every one. Under a saturating workload the adaptive logic was never exercised; the congestion regime was HEAVY for 80.9% of samples and MODERATE for 0.04%. A four-state classifier that occupies two states is not classifying. This is why the present work uses a staged workload in which the flow count changes during the run.

VII. THREATS TO VALIDITY

Emulation. Experiments run under QEMU without hardware virtualisation. We validated that TBF shaping is accurate under emulation before collecting results, and absolute latencies at 20 ms base RTT are dominated by the configured `netem` delay rather than by host timing jitter. Results at much higher link rates, where per-packet processing time matters, would need re-validation.

Scale. A 10 Mbit bottleneck with 8–24 flows is representative of a home or access link, not of a datacentre or backbone. SCRR [10] and QueuePilot [13] both address regimes we do not test.

Traffic model. All flows are bulk TCP CUBIC. Mixed workloads with sparse latency-sensitive flows are exactly where `fq_codel`’s new-flow heuristic is designed to help, and where an adaptive quantum might plausibly matter; we did not test them, and our negative result should not be extrapolated there.

Single bottleneck. Multi-hop topologies with several congested links are not covered.

VIII. CONCLUSION

We set out to determine whether adapting `fq_codel`’s four parameters at runtime improves on its defaults, and built the apparatus to answer it: a testbed with the bottleneck on the data path, a working eBPF flow-telemetry pipeline, and a controller applying Adaptive RED’s AIMD policy under a gradient-based trajectory estimate.

The measured answer, under the conditions we tested, is that it does not. The dominant effect by two orders of magnitude

is the presence or absence of flow queueing with a delay-targeting AQM, not the tuning of that AQM’s parameters. This is consistent with CoDel’s authors’ position that its defaults are RTT-relative by construction.

We think the more useful contribution is the methodological one. Each of the pitfalls in Section VI produced results that were confident, internally consistent and wrong, and each was caught only by checking a physical invariant – bytes per packet, the link’s maximum packet rate, whether a control input changes its output – rather than by inspecting a summary statistic. We suggest those checks as routine practice, and we release the full artefact so that our own numbers can be subjected to them.

REPRODUCIBILITY

All code, kernel configuration, logs and analysis are available at <https://github.com/Amritha902/ccn-linux-qdisc-study>. Every table and every inline figure in this paper is generated directly from the experiment logs by `src/analyse.py`; none is typed by hand.

REFERENCES

- [1] J. Gettys and K. Nichols, “Bufferbloat: Dark Buffers in the Internet,” *Commun. ACM*, vol. 55, no. 1, pp. 57–65, Jan. 2012.
- [2] S. Floyd and V. Jacobson, “Random Early Detection Gateways for Congestion Avoidance,” *IEEE/ACM Trans. Netw.*, vol. 1, no. 4, pp. 397–413, Aug. 1993.
- [3] S. Floyd, R. Gummadi, and S. Shenker, “Adaptive RED: An Algorithm for Increasing the Robustness of RED’s Active Queue Management,” ICSI Technical Report, Aug. 2001.
- [4] K. Nichols and V. Jacobson, “Controlling Queue Delay,” *ACM Queue*, vol. 10, no. 5, pp. 20–34, May 2012.
- [5] K. Nichols, V. Jacobson, A. McGregor, and J. Iyengar, “Controlled Delay Active Queue Management,” IETF RFC 8289, Jan. 2018.
- [6] T. Høiland-Jørgensen, P. McKenney, D. Täht, J. Gettys, and E. Dumazet, “The FlowQueue-CoDel Packet Scheduler and Active Queue Management Algorithm,” IETF RFC 8290, Jan. 2018.
- [7] R. Pan, P. Natarajan, F. Baker, and G. White, “Proportional Integral Controller Enhanced (PIE): A Lightweight Control Scheme to Address the Bufferbloat Problem,” IETF RFC 8033, Feb. 2017.
- [8] G. Ramakrishnan, M. Bhasi, V. Saicharan, L. Monis, S. D. Patil, and M. P. Tahiliani, “FQ-PIE Queue Discipline in the Linux Kernel: Design, Implementation and Challenges,” in *Proc. IEEE LCN Symp.*, Oct. 2019.
- [9] T. Høiland-Jørgensen, D. Täht, and J. Morton, “Piece of CAKE: A Comprehensive Queue Management Solution for Home Gateways,” in *Proc. IEEE LANMAN*, Jun. 2018.
- [10] E. Sharafzadeh, R. Matson, J. Tourrilhes, P. Sharma, and S. Ghorbani, “Self-Clocked Round-Robin Packet Scheduling,” in *Proc. USENIX NSDI*, Apr. 2025.
- [11] F. Rodriguez et al. “DESiRED, Dynamic, Enhanced, and Smart iRED: A P4-AQM with Deep Reinforcement Learning and In-band Network Telemetry,” *Computer Networks*, vol. 244, 2024.
- [12] D. Satish et al. “Distilling Large Language Models for Network Active Queue Management,” *IEEE Trans. Netw.*, 2026.
- [13] M. Dery, O. Krupnik, and I. Keslassy, “QueuePilot: Reviving Small Buffers With a Learned AQM Policy,” in *Proc. IEEE INFOCOM*, 2023.
- [14] M. P. Toopchinezhad and M. Ahmadi, “Machine Learning Approaches for Active Queue Management: A Survey, Taxonomy, and Future Directions,” *Computer Networks*, vol. 262, May 2025.
- [15] J. Ye and K.-C. Leung, “Adaptive and Stable Delay Control for Combating Bufferbloat: Theory and Algorithms,” *IEEE Systems Journal*, vol. 14, no. 1, pp. 1285–1296, Mar. 2020.
- [16] B. Briscoe, K. De Schepper, M. Bagnulo, and G. White, “Low Latency, Low Loss, and Scalable Throughput (L4S) Internet Service: Architecture,” IETF RFC 9330, Jan. 2023.
- [17] K. De Schepper, B. Briscoe, and G. White, “Dual-Queue Coupled Active Queue Management (AQM) for Low Latency, Low Loss, and Scalable Throughput (L4S),” IETF RFC 9332, Jan. 2023.

- [18] S. Ha, I. Rhee, and L. Xu, "CUBIC: A New TCP-Friendly High-Speed TCP Variant," *ACM SIGOPS Oper. Syst. Rev.*, vol. 42, no. 5, pp. 64–74, 2008.
- [19] V. S. Borkar, "Stochastic Approximation with Two Time Scales," *Syst. Control Lett.*, vol. 29, no. 5, pp. 291–294, 1997.
- [20] R. Jain, D.-M. Chiu, and W. Hawe, "A Quantitative Measure of Fairness and Discrimination for Resource Allocation in Shared Computer Systems," DEC Research Report TR-301, Sep. 1984.
- [21] M. A. M. Vieira et al. "Fast Packet Processing with eBPF and XDP: Concepts, Code, Challenges, and Applications," *ACM Comput. Surv.*, vol. 53, no. 1, Article 16, 2020.
- [22] F. Baker and G. Fairhurst, "IETF Recommendations Regarding Active Queue Management," IETF RFC 7567, Jul. 2015.